

PostgreSQL + sqlc

Persistência type-safe para APIs Go

DIM0547 — Sprint 2 (05/05)

Prof. Fernando · UFRN · 2026.1

0 problema

```
var contacts = map[string]Contact{} // Sprint 1
```

- Reinicia o servidor → **perde tudo**
- Não escala para múltiplas instâncias
- Sem transações, sem índices, sem integridade referencial

Sprint 2: trocar por **PostgreSQL** de verdade.

A pergunta é: **como acessar o banco em Go?**

Três abordagens

Abordagem	Exemplo	Prós	Contras
ORM	GORM	Produtividade rápida	Abstrações vazam, queries escondidas
Raw SQL	<code>database/sql</code>	Controle total	Repetitivo, sem type safety
SQL-first codegen	sqlc ✓	SQL real + tipo seguro	Precisa saber SQL

Por que sqlc (e não GORM)

GORM (ORM):

```
db.Where("email = ?", email).First(&contact)
// Que SQL isso gera? 🤖
// E se a query for lenta? Debug no escuro.
```

sqlc (SQL-first):

```
-- name: GetContactByEmail :one
SELECT * FROM contacts WHERE email = $1;
```

```
contact, err := queries.GetContactByEmail(ctx, email)
// SQL visível, tipado, compile-time checked
```

ORM esconde SQL. Quando a query fica lenta, vocês vão debugar SQL de qualquer jeito. Melhor começar sabendo SQL.

Como sqlc funciona

1. Você escreve SQL → 2. sqlc gera Go → 3. Você usa Go tipado

db/schema/001.sql	sqlc generate	internal/db/
CREATE TABLE ...	→	contacts.sql.go
		models.go
db/queries/contacts.sql		db.go
-- name: List :many		
SELECT * FROM ...		

Erros de SQL são pegos em **compile time**, não em runtime.

Setup: sqlc.yaml

```
version: "2"
sql:
  - engine: "postgresql"
    queries: "db/queries/"
    schema: "db/schema/"
    gen:
      go:
        package: "db"
        out: "internal/db"
```

```
go install github.com/sqlc-dev/sqlc/cmd/sqlc@latest
```

Passo 1: Definir o schema

```
-- db/schema/001_contacts.sql

CREATE TABLE contacts (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  email        TEXT NOT NULL UNIQUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

SERIAL = auto-incremento. **TIMESTAMPTZ** = timestamp com timezone. **UNIQUE** = banco garante unicidade, não o código Go.

Passo 2: Escrever as queries

```
-- db/queries/contacts.sql

-- name: ListContacts :many
SELECT * FROM contacts ORDER BY created_at DESC;

-- name: GetContact :one
SELECT * FROM contacts WHERE id = $1;

-- name: CreateContact :one
INSERT INTO contacts (name, email)
VALUES ($1, $2)
RETURNING *;

-- name: DeleteContact :exec
DELETE FROM contacts WHERE id = $1;
```


Anotações sqlc

Anotação	Retorno Go	Quando usar
<code>:one</code>	<code>(Model, error)</code>	SELECT que retorna 1 registro
<code>:many</code>	<code>([]Model, error)</code>	SELECT que retorna N registros
<code>:exec</code>	<code>error</code>	DELETE, UPDATE sem retorno
<code>:execresult</code>	<code>(sql.Result, error)</code>	Quando precisa de RowsAffected
<code>:execrows</code>	<code>(int64, error)</code>	Atalho para RowsAffected

RETURNING * → faz INSERT/UPDATE retornar o registro criado (use com `:one`)

Passo 3: Gerar código

```
sqlc generate
```

Gera automaticamente em `internal/db/`:

```
// models.go – structs
type Contact struct {
    ID          int32
    Name        string
    Email       string
    CreatedAt   time.Time
}

// contacts.sql.go – funções
func (q *Queries) ListContacts(ctx context.Context) ([]Contact, error)
func (q *Queries) GetContact(ctx context.Context, id int32) (Contact, error)
func (q *Queries) CreateContact(ctx context.Context, arg CreateContactParams) (Contact, error)
func (q *Queries) DeleteContact(ctx context.Context, id int32) error
```

Struct gerada com tipos corretos: `int32` para SERIAL, `time.Time` para TIMESTAMPTZ.

Passo 4: Conectar ao banco

```
import (  
    "github.com/jackc/pgx/v5/pgxpool"  
    "meuapp/internal/db"  
)  
  
func main() {  
    ctx := context.Background()  
  
    pool, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))  
    if err != nil {  
        log.Fatal(err)  
    }  
    defer pool.Close()  
  
    queries := db.New(pool)  
  
    router := handler.NewRouter(queries)  
    http.ListenAndServe(":8080", router)  
}
```

DATABASE_URL: postgres://user:pass@localhost:5432/mydb?sslmode=disable

Passo 5: Usar nos handlers

```
func (h *Handler) listContacts(w http.ResponseWriter, r *http.Request) {
    contacts, err := h.queries.ListContacts(r.Context())
    if err != nil {
        writeProblem(w, ProblemDetails{
            Status: 500, Title: "Internal Error",
            Detail: "failed to list contacts",
        })
        return
    }
    writeJSON(w, http.StatusOK, contacts)
}
```

Diferença do map:

- Antes: `list := make([]Contact, 0); for _, c := range a.contacts { ... }`
- Depois: `contacts, err := h.queries.ListContacts(r.Context())`

Mais simples, mais seguro, persistente.

Migrations: versionando o schema

Problema: como aplicar mudanças no banco de forma **reprodutível e versionada**?

Migrations = scripts SQL numerados que transformam o schema:

```
db/schema/  
├─ 001_contacts.sql      # CREATE TABLE contacts  
├─ 002_add_phone.sql     # ALTER TABLE contacts ADD COLUMN phone TEXT  
└─ 003_create_orders.sql # CREATE TABLE orders
```

Cada migration roda **uma vez**, em ordem. O banco lembra quais já foram aplicadas.

Aplicando migrations (simples)

Opção mínima (sem ferramenta):

```
psql -U user -d mydb -f db/schema/001_contacts.sql
```

Com golang-migrate (recomendado para CI):

```
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest  
  
migrate -path db/schema -database "$DATABASE_URL" up
```

Com Atlas (mais poderoso):

```
atlas migrate apply --url "$DATABASE_URL"
```

Para a Sprint 2, **qualquer uma das três opções** é aceitável. O que importa é que o schema esteja versionado em arquivo.

Docker: PostgreSQL local

Para desenvolver, **não instale PostgreSQL na máquina**. Use Docker:

```
docker run -d \  
  --name postgres-dev \  
  -p 5432:5432 \  
  -e POSTGRES_USER=dev \  
  -e POSTGRES_PASSWORD=secret \  
  -e POSTGRES_DB=myapp \  
  postgres:15
```

Ou com **docker-compose.yml** (Sprint 2 pede):

```
services:  
  db:  
    image: postgres:15  
    ports: ["5432:5432"]  
    environment:  
      POSTGRES_USER: dev  
      POSTGRES_PASSWORD: secret  
      POSTGRES_DB: myapp
```

```
docker compose up -d
```

Estrutura do projeto Sprint 2

```
meu-projeto/
├── cmd/api/main.go
├── internal/
│   ├── db/                                ← NOVO (gerado pelo sqlc)
│   │   ├── contacts.sql.go
│   │   ├── db.go
│   │   └── models.go
│   ├── handler/
│   │   └── contacts.go                    ← usa db.Queries
│   └── middleware/
├── db/
│   ├── schema/001_contacts.sql           ← NOVO (migrations)
│   └── queries/contacts.sql              ← NOVO (queries SQL)
├── sqlc.yaml                             ← NOVO
├── docker-compose.yml                    ← NOVO
├── go.mod
└── .github/workflows/ci.yml
```


O que mudou do Sprint 1 para o Sprint 2

Sprint 1	Sprint 2
<code>map[string]Contact</code>	PostgreSQL + sqlc
Dados em memória	Dados persistentes
Sem schema	Schema versionado
Sem Docker	docker-compose
Tudo em <code>handler/</code>	Separação handler ↔ db

Sprint 3 vai separar ainda mais: **Clean Architecture** com camadas service/repository.

Resumo

1. **sqlc** gera Go tipado a partir de SQL puro
2. **Schema** versionado em arquivos SQL (`db/schema/`)
3. **Queries** anotadas com `:one` , `:many` , `:exec`
4. **sqlc generate** cria structs + funções automaticamente
5. **PostgreSQL via Docker** para desenvolvimento local
6. **Migrations** para aplicar schema de forma reprodutível

Para fazer agora:

- Sprint 1: até 30/04 (encerrado)
- Sprint 2: conectar API ao PostgreSQL — entrega **19/05 (ter)**
- Lista 3: ⚠ prazo prorrogado até **12/05 (ter) às 23:59**
- Lista 4 (sqlc + Clean Architecture): publicada quinta 07/05